

ERP RAG Assistant - Design Document

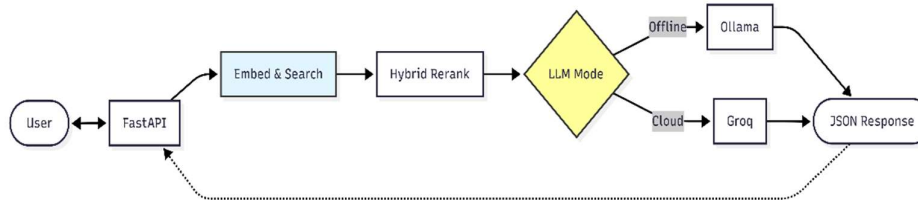
1. Problem Statement

The project addresses the need for a production-grade Retrieval-Augmented Generation (RAG) system capable of querying ERP documentation efficiently. It solves the challenge of providing accurate, source-attributed answers with the flexibility of hybrid inference—functioning fully offline for privacy or utilizing cloud resources for performance.

2. Architecture & Flow

The system follows a layered architecture: UI (Vanilla JS) → API (FastAPI) → Business Logic (RAG) → Data (FAISS/PDFs).

- **Data Ingestion:** PDFs are parsed using PyMuPDF into 200-word chunks with a 20-word overlap to preserve context.
- **RAG Pipeline Flow:**
 1. **Query Embedding:** User input is converted to vector format.
 2. **Vector Search:** FAISS performs a K-Nearest Neighbors (KNN) search to find relevant chunks.
 3. **Hybrid Re-ranking:** Results are re-scored based on relevance and historical user feedback.
 4. **LLM Generation:** The selected context and query are sent to the active LLM (Local or Cloud).
 5. **Response:** The system returns the generated answer with strict source attribution.



3. Tools & Libraries

- **Backend API:** FastAPI + Uvicorn (Async-first, RESTful endpoints).
- **LLM Backends:**
 - *Local:* Ollama + Qwen 2.5 3B (4-bit quantized, ~3GB VRAM, offline).
 - *Cloud:* Groq API + Llama 3.1 8B (High throughput ~141 tok/s).
- **Vector Search:** FAISS (Flat index, <100ms search latency).
- **Embeddings:** SentenceTransformers (all-MiniLM-L6-v2, 384-dim).
- **Ingestion:** PyMuPDF (Fast text extraction).

4. Key Logic Explanation

- **Hybrid Scoring Algorithm:** To improve retrieval quality over time, the system re-ranks retrieved documents using a weighted formula: $\text{Score} = 0.7 * \text{Semantic_Relevance} + 0.3 * \text{Feedback_Score}$.
- **Factory Pattern for Inference:** The application uses a Python factory pattern to instantiate LLM clients at runtime. This allows seamless switching between the OllamaClient (local) and GroqClient (cloud) without restarting the application.
- **Semantic Retrieval:** It utilizes dense vector retrieval where queries are embedded into 384-dimensional space, allowing FAISS to identify semantically similar content rather than just keyword matching.